

Python List Objects

To modify a tuple in Python you must create a new tuple. This is because tuples are readonly collections. If you want a **mutable** collection of objects you should use *lists*.

To create a list in Python, write some Python objects between **brackets** seperating them with `,`.

In [4]:
```
['hello', 'world', 1, None, 1]
```

Out[4]:
```
['hello', 'world', 1, None, 1]
```

Like tuples, Python also allows one more `,` at the end of the list. This extra `,` does not have an effect. The list created will be the same.

In [5]:
```
['hello', 'world', 1, None, 1, ]
```

Out[5]:
```
['hello', 'world', 1, None, 1]
```

Unlike tuples, you can create a list with single object without using extra `,` at the end.

In [6]:
```
['hello']
```

Out[6]:
```
['hello']
```

You can create a list with an iterable using `list()` function.

In [7]:
```
list('Hello world!')
```

Out[7]:
```
['H', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '!']
```

Or you can create a list with comprehension syntax.

In [8]:
```
[x**2 for x in range(10) if x & 1]
```

Out[8]:
```
[1, 9, 25, 49, 81]
```

In comprehension `if` part is optional.

In [9]:
```
[x**2 for x in range(10)]
```

Out[9]:
```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Empty List

An empty list can be created by `[]` or `list()`.

Operations on Lists

Any operation on tuples are also aplicable to the lists in the same way.

Since list is one Python object then you can assign it to a variable.

In [10]:
```
h = ['hello']
w = ['world', 1]
```

In [11]:
```
len(w)
```

Out[11]:
```
2
```

In [12]:
```
'world' in w
```

Out[12]:
```
True
```

In [13]:
```
w.count('hello')
```

Out[13]:
```
0
```

In [14]:
```
h + w
```

Out[14]:
```
['hello', 'world', 1]
```

In [15]:
```
(h + w) * 3
```

Out[15]:
```
['hello', 'world', 1, 'hello', 'world', 1, 'hello', 'world', 1]
```

You can also nest tuples and lists like tuples. Indeed you can nest tuples to lists and lists to tuples.

In [16]:
```
a = ['one', 2, (3, [4, 5])]
```

You can also access the items and ranges of a list like a tuple.

In [17]:
```
a[-1][1][0:-1]
```

Out[17]:
```
[4]
```

Modifying The Lists

In addition to tuples, lists are **mutable**.

Changing An Item of The List

You can change any item by assigning it a different object.

In [18]:
```
a[-1] = "last"
a
```

Out[18]:
```
['one', 2, 'last']
```

You can also change a range of items from a list.

In [19]:
```
a[::2] = [1,1]
a
```

Out[19]:
```
[1, 2, 1]
```

Removing An Item From The List

To remove the object `obj` from a list use `.remove(obj)` method.

In [20]:
```
a.remove(1)
a
```

Out[20]:
```
[2, 1]
```

You can delete an item from a list by `del` keyword.

In [21]:
```
del a[0]
```

Now the first item in the list is removed.

In [22]:
```
a
```

Out[22]:
```
[1]
```

To rewmove all the items from a list use `.clear()` method.

In [23]:
```
a.clear()
a
```

Out[23]:
```
[]
```

Adding New Items To The List

To add an object `obj` at the end of the list use `.append(obj)` method.

In [24]:
```
a.append(8)
a
```

Out[24]:
```
[8]
```

To add multiple objects in a single method use `.extend(iterable)` method with a list `iterable` (or any iterable):

In [25]:
```
a.extend([19,10,11])
a
```

Out[25]:
```
[8, 19, 10, 11]
```

To add an object `obj` at an arbitrary `index` shifting the remaining, use `.insert(index,obj)` method.

In [26]:
```
a.insert(20,-10)
a
```

Out[26]:
```
[8, 19, 10, 11, -10]
```

Here since the index 20 is invalid Python assumed it as last.

Reordering Lists

In [27]:
```
a.sort()
a
```

Out[27]:
```
[-10, 8, 10, 11, 19]
```

In [28]:
```
a.reverse()
a
```

Out[28]:
```
[19, 11, 10, 8, -10]
```

Making A Copy of A List

If you write:

In [29]:
```
b = a
```

Then Python gives just another name to the same list object. If you modify `b` here then `a` is also modified because they are essentially the same. For example:

In [30]:
```
b[0] = None
```

In [31]:
```
a
```

Out[31]:
```
[None, 11, 10, 8, -10]
```

In [32]:
```
b
```

Out[32]:
```
[None, 11, 10, 8, -10]
```

To make a copy of a list use `.copy()` method.

In [33]:
```
b = a.copy()
```

Now `a` and `b` are different list objects.

In [34]:
```
del a[0]
```

In [35]:
```
a
```

Out[35]:
```
[11, 10, 8, -10]
```

In [36]:
```
b
```

Out[36]:
```
[None, 11, 10, 8, -10]
```

Also range operations create a new list. So writing:

In [37]:
```
b = a[:]
```

will also make a copy of the list `a`.